

```

"use strict";

const fs = require("fs");
const crypto = require("crypto");
const express = require("express");
const pino = require("pino");
const baileys =
require("@whiskeysockets/baileys");

const makeWASocket = baileys.default ||
baileys.makeWASocket;
const {
  useMultiFileAuthState,
  DisconnectReason,
  downloadMediaMessage,
  jidNormalizedUser,
  fetchLatestBaileysVersion
} = baileys;

/* ----- Config (Replit Secrets) -----
----- */

const PORT = Number(process.env.PORT) ||
3000;
const OWNER_DIGITS =
String(process.env.OWNER_NUMBER ||
 "").replace(/\D/g, "");
const PAIRING_DIGITS =
String(process.env.PAIRING_NUMBER ||
 "").replace(/\D/g, "");

```

```

const DASHBOARD_PASSWORD =
process.env.DASHBOARD_PASSWORD || "";

const AUTH_DIR = "./auth";
const COOLDOWN_MS = 30 * 1000;
const MAX_MESSAGE_AGE_SEC = 120;

if (!DASHBOARD_PASSWORD) {
  console.error("DASHBOARD_PASSWORD Secret
ma jiro. Ku dar Replit Secrets kadibna dib
u bilow.");
  process.exit(1);
}
if (!OWNER_DIGITS) {
  console.warn("OWNER_NUMBER Secret ma
jiro: forwarding wuxuu u dhaqmi doonaa
akoonka bot-ka laftiisa.");
}

/* ----- State ----- */

let sock = null;
let starting = false;
let connecting = false;
let connected = false;
let forwardingEnabled = true;
let autoReplyEnabled = true;
let waitingForPairing = false;
let lastError = null;
let startedAt = null;
let reconnectTimer = null;

const cooldowns = new Map();
const logger = pino({ level: "silent" });

```

```
/* ----- Somali auto reply -----  
*/  
  
const replyRules = [  
  { words: ["asc", "aslm",  
    "assalamualaikum", "assalamu calaykum",  
    "salaan", "salam"], replies: ["wcs",  
    "wcsalaam", "waaleykum salaam"] },  
  { words: ["sidee tahay", "see tahay",  
    "sidee thy", "sthy", "seithy", "sidee  
    ahaan"], replies: ["fcn alhamdulilaah adiga  
    sidee tahay", "waan fiicanahay adiga",  
    "nabad alhamdulilaah adiga"] },  
  { words: ["nabad miyaa", "nabad", "nabad  
    baa"], replies: ["nabad haa", "nabad  
    alhamdulilaah", "haa nabad"] },  
  { words: ["subax wanaagsan", "subax  
    wanaag"], replies: ["subax wanaagsan",  
    "subax nabad ah", "subax wanaag adiga"] },  
  { words: ["galab wanaagsan", "galab  
    wanaag"], replies: ["galab wanaagsan",  
    "galab nabad ah"] },  
  { words: ["habeen wanaagsan", "habeen  
    wanaag"], replies: ["habeen wanaagsan",  
    "hurdo wanaagsan", "habeen nabad ah"] },  
  { words: ["maalin wanaagsan"], replies:  
    ["maalin wanaagsan adiguna", "mahadsanid  
    adiguna"] },  
  { words: ["xagee joogtaa", "xaggee  
    joogtaa", "xage joogtaa", "xagee tahay"],  
    replies: ["magaalada joogaa", "guriga  
    joogaa", "halkaan joogaa"] },  
  { words: ["waryaa", "war yaa", "war"],
```

```
replies: ["hee waryaa", "haye waan ku
dhegeysanayaa", "waan joogaa waryaa"] },
  { words: ["maxaa cusub", "maxaa cusub
aa", "maxaa soo kordhay", "wax cusub"],
replies: ["wax cusub ma jiro", "wax cusub
ma jiraan adiga", "nabad kuma jirto wax
cusub"] },
  { words: ["maxaa dhacay", "maxaa ka
dhacay"], replies: ["waxba ma dhicin",
"nabad wax xun ma jiraan"] },
  { words: ["maxaa wadaa", "maxaad wadaa",
"maxa wadaa", "maxaad qabanaysaa"],
replies: ["iska joogaa adiga", "waxba iska
joogaa", "wax gaar ah ma wado"] },
  { words: ["maxaa kuu socda", "maxaa kuu
jira"], replies: ["wax badan ma socdaan",
"nabad adiga"] },
  { words: ["ma joogtaa", "ma joogtid",
"joogtaa", "joogtid"], replies: ["haa waan
joogaa", "haa halkaan ayaan joogaa"] },
  { words: ["hurudaa", "ma hurudaa",
"hurudaa miyaa", "ma seexatay"], replies:
["maya waan joogaa", "maya ma hurdo"] },
  { words: ["yaa tahay", "yaa tahay adiga",
"ka yaa tahay", "yaad tahay"], replies:
["aniga Abdullahi Bot", "waxaan ahay
Abdullahi Bot"] },
  { words: ["ok", "okay", "okey", "haye",
"hayaa", "hayye"], replies: ["haye",
"hayaa", "fiican"] },
  { words: ["mahadsanid", "mahad sanid",
"thanks", "thx", "thank you", "mahadsan"],
replies: ["waa ku mahadsan tahay", "wax
kasta ma jiraan", "adigaa mahadsan"] },
```

```
{ words: ["waan ku mahadsanahay", "aad
iyo aad ayaad mahadsantahay"], replies:
["wax kasta ma jiraan", "waa ku soo
dhawaaday"] },
{ words: ["nabad gelyo", "bye",
"nabadgelyo", "is arag"], replies: ["nabad
gelyo", "is arag", "nabad gelyo walaal"] },
{ words: ["soo dhawoow", "soo dhawow"],
replies: ["mahadsanid", "waad mahadsan
tahay"] },
{ words: ["ma cunay", "cuntay", "ma
cuntay"], replies: ["haa waan cunay adiga",
"wali ma cunin adiga"] },
{ words: ["ma cabtay shaah", "shaah"],
replies: ["haa waan cabay", "wali ma
cabin"] },
{ words: ["xaalkaa", "xaalku", "xaalka",
"xaalkaaga", "xaaladu sidee tahay"],
replies: ["fiican alhamdulilaah",
"wanaagsan adiga", "waan fiicanahay"] },
{ words: ["ma fiicantahay",
"fiicantahay", "ma fiicnaatay"], replies:
["haa waan fiicanahay", "fiican
alhamdulilaah"] },
{ words: ["ma nabad baa", "nabad ma
tahay"], replies: ["haa nabad", "nabad
alhamdulilaah"] },
{ words: ["walaal", "walaalo", "wiil",
"adeer", "abti"], replies: ["hee walaal",
"haye walaal", "waan ku dhegeysanayaa"] },
{ words: ["mudane", "marwo"], replies:
["haye", "waan ku dhegeysanayaa"] },
{ words: ["ma shaqeynaysaa", "shaqo ma
leedahay"], replies: ["haa waan
```

```
shaqeynayaa", "haa waan mashquulsanahay"]
},
{ words: ["ma mashquulsantahay",
"mashquul ma tahay"], replies: ["yara ayaan
mashquulsanahay", "maya waan firaaqo ahay"]
},
{ words: ["ma diyaar tahay", "diyaar ma
tahay"], replies: ["haa diyaar", "haa waan
diyaar ahay"] },
{ words: ["ma imaanaysaa", "ma soo
socotaa"], replies: ["waan imaanayaa",
"wali ma hubo"] },
{ words: ["ma degan tahay", "degan
tahay"], replies: ["haa waan degan ahay",
"haa nabad"] },
{ words: ["haa", "haa waa sax", "sax"],
replies: ["hayee", "waa sax", "hayaa"] },
{ words: ["maya", "maya maya"], replies:
["hayee", "waa fiican", "ok"] },
{ words: ["iga raali ahow", "raali ahow",
"raali noqo"], replies: ["waa fiican", "wax
jirin", "waa la iska raalin yahay"] },
{ words: ["wax xun ma jiraan", "wax xun
ma jiro"], replies: ["wax xun ma jiraan",
"nabad alhamdulilaah"] },
{ words: ["waa sidee", "sidee waaye",
"sidee yahay"], replies: ["fiican", "waa
nabad", "waa wanaagsan"] },
{ words: ["i soo wac", "soo wac"],
replies: ["hadhow ayaan ku soo wacayaa",
"yara sug"] },
{ words: ["sug", "yara sug", "hal
daqiiqo"], replies: ["hayee waan sugayaa",
"hayaa"] },
```

```
{ words: ["hadaba", "hada"], replies:
["haye", "hayaa waan ku dhegeysanayaa"] },
{ words: ["maxaa tiri", "maxaad tiri",
"maxaa la yidhi"], replies: ["waxba ma
odhan", "waan ku celinayaa"] },
{ words: ["ma la socotaa", "ma
maqashay"], replies: ["haa waan maqlay",
"maya wali ma maqlin"] },
{ words: ["waan ku jeclahay"], replies:
["mahadsanid", "adigaa mahadsan"] },
{ words: ["wanaagsan", "fiican", "waa
fiican"], replies: ["haye",
"alhamdulilaah", "hayaa"] },
{ words: ["alhamdulilaah",
"alxamdulillaah", "alhamdulillah"],
replies: ["alhamdulilaah", "alhamdulilaah
walaal"] },
{ words: ["inshallah", "insha allah",
"inshaa allaah"], replies: ["inshaa
allaah", "aamiin"] },
{ words: ["aamiin", "amiin"], replies:
["aamiin", "aamiin walaal"] },
{ words: ["ma jiraa", "ma jirtaa"],
replies: ["haa jiraa", "haa waan jiraa"] },
{ words: ["waa kuma", "kuma tahay"],
replies: ["waa aniga Abdullahi Bot", "aniga
ayaa ah"] },
{ words: ["maxaa la sameeyaa", "maxaan
sameeyaa"], replies: ["sug wax yar", "iska
sug"] },
{ words: ["xaggee ku kulannaa", "xagee ku
kulannaa"], replies: ["waan ku soo
sheegayaa", "hadhow ayaan ku soo
sheegayaa"] },
```

```

    { words: ["waqtigee", "goorma", "imisa"],
      replies: ["wali ma hubo", "waan ku soo
sheegayaa"] },
    { words: ["hello", "hi", "hey", "hii",
"hallo"], replies: ["hee", "haye", "hello"]
},
    { words: ["ma la hadli karaa", "ma hadli
karnaa"], replies: ["haa waan joogaa", "haa
hadal"] },
    { words: ["maxaa yahay", "maxay tahay"],
replies: ["waxba ma jiraan", "nabad
walaal"] },
    { words: ["ka warran", "ka warran
walaal"], replies: ["nabad alhamdulilaah",
"fiican adiga"] }
];

const replyLookup = new Map();
for (const rule of replyRules) {
  for (const w of rule.words) {
    const key = normalizeText(w);
    if (key && !replyLookup.has(key))
      replyLookup.set(key, rule.replies);
  }
}

function normalizeText(text) {
  return String(text || "")
    .toLowerCase()
    .replace(/[\p{L}\p{N}\s]/gu, "")
    .replace(/\s+/g, " ")
    .trim();
}

```

```

function cleanReply(text) {
  return normalizeText(text);
}

function randomItem(arr) {
  return arr[Math.floor(Math.random() *
arr.length)];
}

function getAutoReply(messageText) {
  const key = normalizeText(messageText);
  if (!key) return null;
  const replies = replyLookup.get(key);
  if (!replies) return null;
  return cleanReply(randomItem(replies));
}

function cooldownActive(jid) {
  const last = cooldowns.get(jid);
  return Boolean(last && Date.now() - last
< COOLDOWN_MS);
}

setInterval(() => {
  const now = Date.now();
  for (const [jid, t] of cooldowns) {
    if (now - t > COOLDOWN_MS)
cooldowns.delete(jid);
  }
}, 60 * 1000).unref();

/* ----- Message helpers ----- */

function unwrap(msg) {

```

```

    let cur = msg;
    for (let i = 0; i < 5 && cur; i++) {
        const next =
cur.ephemeralMessage?.message ||
cur.documentWithCaptionMessage?.message;
        if (!next) break;
        cur = next;
    }
    return cur;
}

```

// View Once is only DETECTED so it can be ignored. It is never downloaded or recovered.

```

function isViewOnce(msg) {
    if (!msg) return false;
    const c = unwrap(msg);
    return Boolean(
        msg.viewOnceMessage ||
        msg.viewOnceMessageV2 ||
        msg.viewOnceMessageV2Extension ||
        c?.viewOnceMessage ||
        c?.viewOnceMessageV2 ||
        c?.viewOnceMessageV2Extension ||
        c?.imageMessage?.viewOnce ||
        c?.videoMessage?.viewOnce ||
        c?.audioMessage?.viewOnce
    );
}

```

```

function getMediaType(content) {
    if (!content) return null;
    if (content.imageMessage) return "image";
    if (content.videoMessage) return "video";
}

```

```
    if (content.audioMessage) return "audio";
    if (content.documentMessage) return
"document";
    return null;
}

function getText(content) {
    if (!content) return "";
    return content.conversation ||
content.extendedTextMessage?.text || "";
}

function getContextInfo(content) {
    return
content?.extendedTextMessage?.contextInfo
|| null;
}

function isIgnoredJid(jid) {
    return (
        !jid ||
        jid.endsWith("@g.us") ||
        jid.endsWith("@broadcast") ||
        jid === "status@broadcast" ||
        jid.endsWith("@newsletter")
    );
}

function senderDigitsList(key) {
    return [key.remoteJid, key.remoteJidAlt,
key.participant, key.participantAlt]
        .filter(Boolean)
        .map((j) => String(j).split("@")
[0].split(":")[0].replace(/\D/g, ""));
}
```

```

}

function isOwnerMessage(key) {
  if (key.fromMe) return true;
  if (!OWNER_DIGITS) return false;
  return
senderDigitsList(key).includes(OWNER_DIGITS
);
}

function ownerJid() {
  if (OWNER_DIGITS) return
`${OWNER_DIGITS}@s.whatsapp.net`;
  if (sock?.user?.id) return
jidNormalizedUser(sock.user.id);
  return null;
}

async function notifyOwner(text) {
  try {
    const jid = ownerJid();
    if (!sock || !connected || !jid)
return;
    await sock.sendMessage(jid, { text });
  } catch (error) {
    lastError = error.message;
    console.error("Notify error:",
error.message);
  }
}

/* ----- Media forwarding (normal
media only) ----- */

```

```

async function forwardMediaToOwner(key,
content, caption) {
  const jid = ownerJid();
  if (!sock || !connected || !jid) return
false;

  const mediaType = getMediaType(content);
  if (!mediaType) return false;

  try {
    const buffer = await
downloadMediaMessage(
      { key, message: content },
      "buffer",
      {},
      { logger, reuploadRequest:
sock.updateMediaMessage }
    );

    if (mediaType === "image") {
      await sock.sendMessage(jid, { image:
buffer, caption: caption || undefined });
    } else if (mediaType === "video") {
      await sock.sendMessage(jid, { video:
buffer, caption: caption || undefined });
    } else if (mediaType === "audio") {
      await sock.sendMessage(jid, {
        audio: buffer,
        mimetype:
content.audioMessage?.mimetype ||
"audio/mp4",
        ptt:
Boolean(content.audioMessage?.ptt)
      });
    }
  }
}

```

```

    } else if (mediaType === "document") {
      await sock.sendMessage(jid, {
        document: buffer,
        mimetype:
content.documentMessage?.mimetype ||
"application/octet-stream",
        fileName:
content.documentMessage?.fileName || "file"
      });
    }
    return true;
  } catch (error) {
    lastError = error.message;
    console.error("Media forwarding
error:", error.message);
    return false;
  }
}

/* ----- Owner-only commands: .vv and
send ----- */

async function handleOwnerCommand(message,
command, content) {
  const ctx = getContextInfo(content);
  const quotedRaw = ctx?.quotedMessage;

  if (!quotedRaw) {
    await notifyOwner(
      command === "send"
        ? "reply garee status media caadi
ah kadib send qor"
        : "reply garee media caadi ah kadib
.vv qor"
    );
  }
}

```

```

    );
    return;
}

if (isViewOnce(quotedRaw)) {
    await notifyOwner("View Once lama
bypass gareyn karo");
    return;
}

const quoted = unwrap(quotedRaw);

if (!getMediaType(quoted)) {
    await notifyOwner("fariinta aad reply
gareysay media ma aha");
    return;
}

const quotedKey = {
    remoteJid: ctx.remoteJid ||
message.key.remoteJid,
    id: ctx.stanzaId,
    participant: ctx.participant,
    fromMe: false
};

const caption = command === "send" ?
"Status media caadi ah" : "Media caadi ah";
const ok = await
forwardMediaToOwner(quotedKey, quoted,
caption);
    await notifyOwner(ok ? "waa la diray" :
"media lama diri karin");
}

```

```

/* ----- Incoming messages -----
*/

async function handleIncoming(message) {
  const key = message?.key;
  if (!key || !message.message) return;
  if (isIgnoredJid(key.remoteJid)) return;

  const ts =
    Number(String(message.messageTimestamp ||
    0));
  if (ts && Date.now() / 1000 - ts >
    MAX_MESSAGE_AGE_SEC) return;

  // View Once is deliberately ignored
  everywhere
  if (isViewOnce(message.message)) return;

  const content = unwrap(message.message);
  const text = getText(content);
  const command =
    text.trim().toLowerCase().replace(/^\./,
    "");
  const owner = isOwnerMessage(key);

  if (command === "vv" || command ===
  "send") {
    // Only the owner may use commands;
    everyone else is silently ignored
    if (owner) await
    handleOwnerCommand(message, command,
    content);
    return;
  }

```

```

    }

    // Own messages never trigger auto reply
    or forwarding
    if (key.fromMe || owner) return;

    if (forwardingEnabled &&
        getMediaType(content)) {
        await forwardMediaToOwner(key, content,
            "Fariin cusub");
    }

    if (!autoReplyEnabled || !text) return;
    if (cooldownActive(key.remoteJid))
        return;

    const reply = getAutoReply(text);
    if (!reply) return;

    cooldowns.set(key.remoteJid, Date.now());
    await new Promise((resolve) =>
        setTimeout(resolve, 500 + Math.random() *
            1000));
    await sock.sendMessage(key.remoteJid, {
        text: reply });
}

/* ----- Connection lifecycle -----
-- */

function destroySocket() {
    if (!sock) return;
    const old = sock;
    sock = null;
}

```

```
    connected = false;
    for (const ev of ["connection.update",
"creds.update", "messages.upsert"]) {
      try {
        old.ev.removeAllListeners(ev);
      } catch (_) {}
    }
    try {
      old.end(undefined);
    } catch (_) {}
  }

function scheduleReconnect(ms) {
  if (reconnectTimer) return;
  reconnectTimer = setTimeout(() => {
    reconnectTimer = null;
    startBot();
  }, ms);
}

async function startBot() {
  if (starting) return;
  starting = true;

  if (reconnectTimer) {
    clearTimeout(reconnectTimer);
    reconnectTimer = null;
  }

  try {
    destroySocket();
    lastError = null;
    connecting = true;
    waitingForPairing = false;
```

```
    const { state, saveCreds } = await
useMultiFileAuthState(AUTH_DIR);

    let version;
    try {
        ({ version } = await
fetchLatestBaileysVersion());
    } catch (_) {}

    const s = makeWASocket({
        auth: state,
        logger,
        version,
        printQRInTerminal: false,
        markOnlineOnConnect: false,
        generateHighQualityLinkPreview:
false,
        syncFullHistory: false,
        browser: ["Ubuntu", "Chrome",
"22.04.4"]
    });
    sock = s;

    let pairingRequested = false;

    s.ev.on("creds.update", saveCreds);

    s.ev.on("connection.update", async
(update) => {
        if (s !== sock) return;
        const { connection, lastDisconnect,
qr } = update;
```

```

        // Pairing code: requested once,
        printed to the console only
        if (qr &&
            !s.authState.creds.registered &&
            PAIRING_DIGITS && !pairingRequested) {
            pairingRequested = true;
            waitingForPairing = true;
            try {
                const code = await
s.requestPairingCode(PAIRING_DIGITS);

                console.log("=====
                ==");
                console.log("PAIRING CODE:",
                code);

                console.log("=====
                ==");

                } catch (error) {
                    pairingRequested = false;
                    lastError = error.message;
                    console.error("Pairing error:",
error.message);
                }
            }

            if (connection === "open") {
                connected = true;
                connecting = false;
                waitingForPairing = false;
                startedAt = new
Date().toISOString();
                console.log("WhatsApp connected");
            }

```

```

    if (connection === "close") {
        connected = false;
        connecting = false;
        waitingForPairing = false;

        const statusCode =
lastDisconnect?.error?.output?.statusCode;
        console.log("WhatsApp
disconnected", statusCode);

        if (statusCode ===
DisconnectReason.loggedOut) {
            lastError = "Logged out. Auth waa
la tirtiray, riix START si aad mar kale u
xirto.";
            destroySocket();
            try {
                fs.rmSync(AUTH_DIR, {
recursive: true, force: true });
            } catch (_) {}
        } else {
            scheduleReconnect(statusCode ===
DisconnectReason.restartRequired ? 500 :
5000);
        }
    }
});

s.ev.on("messages.upsert", async ({
messages, type }) => {
    if (s !== sock || type !== "notify")
return;
    for (const message of messages) {

```

```

        try {
            await handleIncoming(message);
        } catch (error) {
            lastError = error.message;
            console.error("Message error:",
error.message);
        }
    }
});
} catch (error) {
    connecting = false;
    connected = false;
    lastError = error.message;
    console.error("Start error:",
error.message);
    scheduleReconnect(5000);
} finally {
    starting = false;
}
}

/* ----- Dashboard + API (password
protected) ----- */

const app = express();
app.disable("x-powered-by");
app.set("trust proxy", true);

function safeEqual(a, b) {
    const ha =
crypto.createHash("sha256").update(String(a
)).digest();
    const hb =
crypto.createHash("sha256").update(String(b

```

```

)).digest();
    return crypto.timingSafeEqual(ha, hb);
}

const authFailures = new Map();

function dashboardAuth(req, res, next) {
    const ip = req.ip || "unknown";
    const rec = authFailures.get(ip);
    if (rec && rec.count >= 10 && Date.now() - rec.first < 10 * 60 * 1000) {
        return res.status(429).send("Too many attempts");
    }

    const challenge = () => {
        res.setHeader("WWW-Authenticate",
            'Basic realm="Bot Dashboard"');
        return
        res.status(401).send("Authentication required");
    };

    const header = req.headers.authorization || "";
    if (!header.startsWith("Basic ")) return challenge();

    let decoded = "";
    try {
        decoded =
        Buffer.from(header.substring(6),
            "base64").toString("utf8");
    } catch (_) {

```

```

    return challenge();
  }

  const password =
    decoded.slice(decoded.indexOf(":") + 1);

  if (!safeEqual(password,
    DASHBOARD_PASSWORD)) {
    const now = Date.now();
    if (!rec || now - rec.first > 10 * 60 *
    1000) authFailures.set(ip, { count: 1,
    first: now });
    else rec.count += 1;
    return challenge();
  }

  authFailures.delete(ip);
  next();
}

app.use(dashboardAuth);
app.use(express.json({ limit: "10kb" }));

app.use((req, res, next) => {
  if (req.method === "POST" &&
    !String(req.headers["content-type"] ||
    "").startsWith("application/json")) {
    return res.status(415).json({ ok:
    false, error: "JSON only" });
  }
  next();
});

app.get("/api/status", (req, res) => {

```

```
res.json({
  connected,
  connecting,
  forwardingEnabled,
  autoReplyEnabled,
  waitingForPairing,
  startedAt,
  lastError
});
});

app.post("/api/forwarding", (req, res) => {
  forwardingEnabled =
Boolean(req.body?.enabled);
  res.json({ ok: true, forwardingEnabled
});
});

app.post("/api/autoreply", (req, res) => {
  autoReplyEnabled =
Boolean(req.body?.enabled);
  res.json({ ok: true, autoReplyEnabled });
});

app.post("/api/start", (req, res) => {
  startBot();
  res.json({ ok: true, message: "Bot start
ayaa la dalbaday" });
});

app.get("/", (req, res) => {
  res.type("html").send(`<!DOCTYPE html>
<html lang="so">
<head>
```

```
<meta charset="UTF-8">
<meta name="viewport"
content="width=device-width,initial-
scale=1">
<title>Somali WhatsApp Bot</title>
<style>
body{margin:0;font-family:Arial,sans-
serif;background:#f3f4f6;color:#111827}
.container{max-
width:500px;margin:auto;padding:20px}
.card{background:#fff;border-
radius:18px;padding:20px;margin-
bottom:15px;box-shadow:0 5px 20px
rgba(0,0,0,.08)}
h1{margin-top:0}
.status{font-size:18px;font-
weight:bold;margin:10px 0}
button{width:100%;padding:13px;margin-
top:10px;border:0;border-radius:12px;font-
size:16px;cursor:pointer}
.on{background:#16a34a;color:#fff}
.off{background:#dc2626;color:#fff}
.start{background:#111827;color:#fff}
.small{color:#6b7280;font-size:14px}
</style>
</head>
<body>
<div class="container">
  <div class="card">
    <h1>Somali WhatsApp Bot</h1>
    <div id="status"
class="status">Loading...</div>
    <div id="error" class="small"></div>
  </div>
```

```

<div class="card">
  <h3>Auto Reply</h3>
  <div id="autoreply"></div>
  <button class="on"
onclick="post('/api/autoreply',
{enabled:true})">ON</button>
  <button class="off"
onclick="post('/api/autoreply',
{enabled:false})">OFF</button>
</div>
<div class="card">
  <h3>Media Forwarding</h3>
  <div id="forwarding"></div>
  <button class="on"
onclick="post('/api/forwarding',
{enabled:true})">ON</button>
  <button class="off"
onclick="post('/api/forwarding',
{enabled:false})">OFF</button>
</div>
<div class="card">
  <h3>Bot Connection</h3>
  <button class="start"
onclick="post('/api/start',{})">START /
CONNECT</button>
  <p class="small">Pairing Code-ka waxaa
laga arki karaa Replit Console-ka oo
kaliya.</p>
</div>
</div>
<script>
async function load(){
  try{
    const r = await fetch("/api/status");

```

```

const s = await r.json();

document.getElementById("status").textContent =
    s.connected ? "Connected" :
    s.waitingForPairing ? "Pairing code" :
    s.eegConsole ? "eeg Console-ka" :
    s.connecting ? "Connecting..." :
    "Offline";

document.getElementById("autoreply").textContent = s.autoReplyEnabled ? "Auto Reply ON" : "Auto Reply OFF";

document.getElementById("forwarding").textContent = s.forwardingEnabled ? "Forwarding ON" : "Forwarding OFF";

document.getElementById("error").textContent = s.lastError ? "Error: " + s.lastError :
    "";
    }catch(e){

document.getElementById("status").textContent = "Dashboard error";
    }
}
async function post(url, body){
    try{
        await fetch(url,{method:"POST",headers:
{"Content-
Type":"application/json"},body:JSON.stringify(body)});
    }catch(e){}

```

```

    load();
  }
  load();
  setInterval(load,3000);
</script>
</body>
</html>`);
});

app.use((err, req, res, next) => {
  console.error("Express error:",
err.message);
  res.status(500).json({ ok: false, error:
"Server error" });
});

process.on("unhandledRejection", (reason)
=> {
  lastError = String(reason?.message ||
reason);
  console.error("Unhandled rejection:",
lastError);
});

process.on("uncaughtException", (error) =>
{
  lastError = error.message;
  console.error("Uncaught exception:",
error.message);
});

app.listen(PORT, "0.0.0.0", () => {
  console.log(`Dashboard running on
0.0.0.0:${PORT}`);
});

```

```
startBot();  
});
```